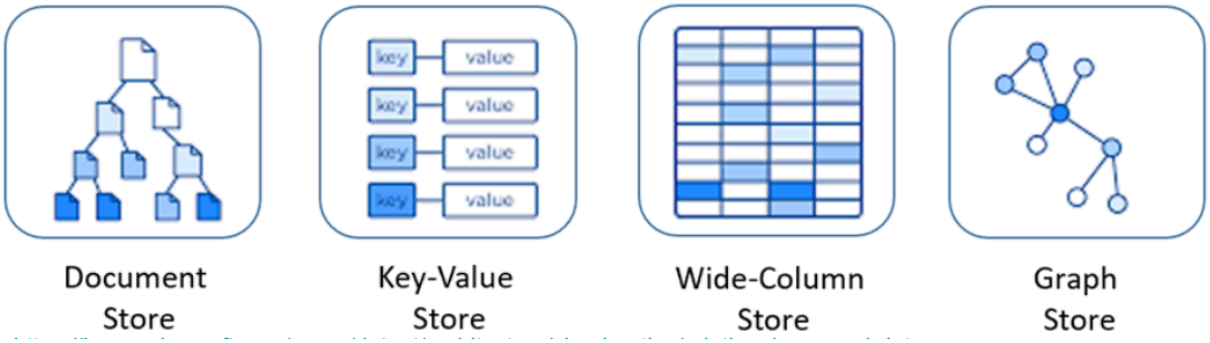
SQL

NoSQL

- İlişkileri olan tablolar olarak yapılandırılmamışlardır
- Daha esneklerdir.



- Graph'ta noktalar düğüm, çizgiler bağlantılar/ilişkilerdir.

RDMS'in ACID Kuralları

- Atomicity (Atomiklik): İşlemler ya tam başarılı ya da tam başarısız olur, yarım kalan işlem olmaz.
- Consistency (Tutarlılık): Veritabanı her işlem sonrasında doğruluk kurallarına uymalıdır.

- Isolation (İzolasyon): Aynı anda yapılan işlemler birini etkilememelidir.
- Durability (Dayanıklılık): Bir işlem tamamlandıktan sonra sistem arızası bile olsa bunu geri alamamalıdır.

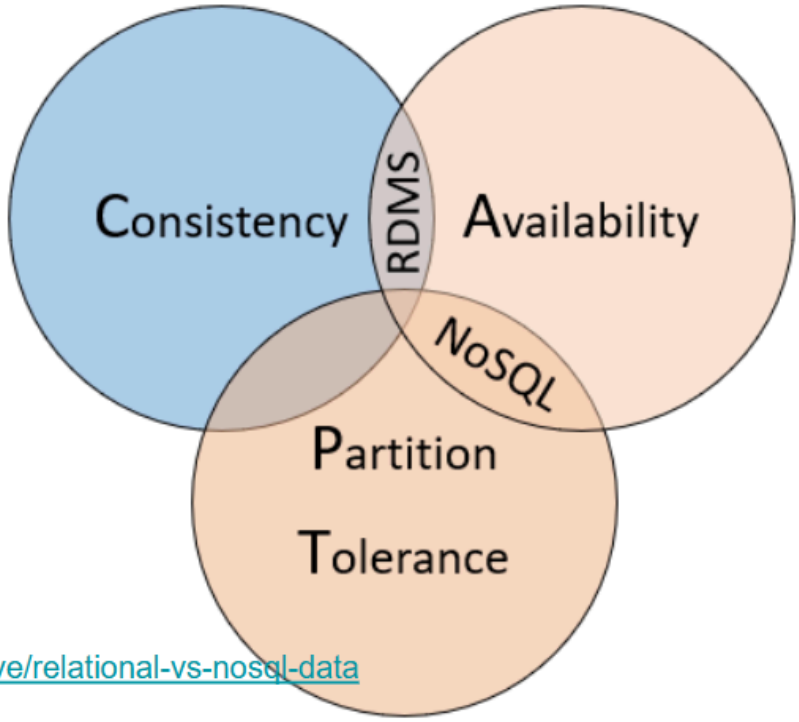
Cap Teoremi

1. Consistency (Tutarlılık): Veritabanı her zaman doğru ve güncel verileri sunmalıdır.
2. Availability (Erişilebilirlik): Veritabanı her zaman erişilebilir olmalıdır.
3. Partition Tolerance (Bölüm Toleransı): Veritabanı ağ bölünmeleri sırasında bile çalışmaya devam edebilmelidir.

- CAP teoremine göre veritabanı sistmeleri bu 3 özellikten aynı anda ikisini sağlayabilir.

SQL vs NoSQL

- SQL ACID özelliklerine sıkı sıkıya bağlıdır.
- Aşağıda görüldüğü üzere CAP teoreminde RDMS tutarlılık ve erişilebilirliğe yönelirken NoSQL Erişilebilirlik ve bölüm toleransına ağırlık verir.



ative/relational-vs-nosql-data

- https://www.youtube.com/watch?v=L86eqtdC2as&t=40s
- Asterisk işareti (*) SQL dilinde bütün anlamında kullanılır.
- SQL'de kod yazarken büyük küçük ayrımı yoktur ancak değişken ismi verirken vardır.
- Metinsel ifadeler tek tırnak içinde gösterilir.

▼ SQL Veri Türleri

▼ Char ve Varchar

- Char ve varchar özünde string türünde veri tutan türlerdir.
- Char(n) şeklinde oluşturduğumuz bir değişken her zaman bellekte n byte alan kaplarken;
- Varchar(n) şeklinde kurduğumuz bir değişken başta n byte tutarken girdiğimiz değerin büyüklüğüne göre alan küçülebilir.

> Örneğin Varchar(20) şeklinde oluşturduğumuz değişken başlangıçta 20 byte yer tutsa bile içine biz 'Tom' değerini girdiğimizde kendimi 3

> byte'ye çeker. Yani girdiğimiz 20 max değer olarak düşünülebilir.

▼ Date

- date türünde veriler 'yıl-ay-gün' şeklinde yazılır.
- Bunlar integer değil string olduğu için tek tırnakta yazılmalıdır.

▼ Veri Tabanı Oluşturma (Create Database)

- Veri tabanları bizim evrensel kümelerinizdir ve yapacağımız tablolar ve gireceğimiz veriler bu veri tabanlarına işlenir, her şeyden önce bir veri tabanı oluşturmamız gerekir.

```
create database mydata #Mydata adında veri tabanı oluşturuldu.
```

- Veri tabanı oluşturduktan sonra işlem yapabilmek için bu veri tabanı seçilmelidir.

```
use mydata #database changed (selected)
```

▼ Tablo Oluşturma (Create Table)

```
create table Makarnalar (Marka nvarchar(50),Fiyat decimal(18,0))
```

▼ Tablolara Ekleme (Insert Function)

```
insert into <Table_name> (Value_Name1,Value_Name2) Values (Value1,Val
ue2);
   #For example:
   insert into Makarnalar (marka,fiyat) Values ('Cingöz',10TL);
```

▼ Tablo Güncelleme (Update Function)

```
update Makarnalar set [fiyat]=15 Where [Marka]='Fırıldak';
```

- Hocanın slaytında köşeli parantez olsa da bu SQL Server için geçerlidir.

- Eğer MySQL kullanılyorsan bu köşeli parantezler yerine geri tırnak (``) kullanılması gereklidir.
- Boş şekilde yazarsan da aynı işlemi görüyor.

```
update Makarnalar set `fiyat`=15 Where `Makarna`='Fırıldak';
update Makarnalar set fiyat=15 Where Makarna='Fırıldak';
```

▼ Tabloda Silme İşlemleri

▼ Delete Function

```
delete from Table_Name where Sütun='Silinecek Öge'
delete from Makarnalar where Marka='Filiz'
```

▼ Drop Function

```
drop table Makarnalar;   #Tablo tamamen silinir.
drop database mydata; #Database'yi komple siler.
```

▼ Select Function

```
select sutun_name from Table_Name #Genel kullanım formatı.

select * from Table_Name #Belirtilen tablodaki bütün sütunları gösterir.

select Name, City from Table_Name #Belirtilen tablodaki İsim ve Şehir sütu
nlarını
                                                    #gösterir.

select * from Table_Name where City='İstanbul' and Postal_Code > 40000
                                        #Şehir istanbul olan ve posta kodu 4000'de
n
                                        #büyük olanları gösterir. Satırların bütün
                                        #sütunlarını gösterir. ("and" istenildiği kdr a
rtabilir.)
```

▼ Not Null

- not null ifadesi ile bu bölümün boş bırakılamaması sağlanır
-

```
create table Pastalar (
Name varchar(20) not null, #İsim boş bırakılamaz.
Size integer              #Büyüklük boş bırakılabilir.
);
```

▼ Primary Key

- Primary key ile girilen değerin hiçbir zaman aynı sütunun başka satırıyla aynı olmaması sağlanır.
- Primary key olarak belirtilen sütunlar hiçbir zaman boş bırakılamaz (default değer belirlenip her seferinde otomatik arttırılması sağlanmadığı sürece.) (not null görevi görür yani.)

```
create table customers (custID integer primary key);
```

▼ Diğer Komutlar:

- ALTER DATABASE: Database düzeyinde değişiklik yapar.
- ALTER TABLE: Tablonun özellikleri değiştirilir.
- CREATE INDEX: Bir dizin oluşturur. (Arama anahtarı.)
- DROP INDEX: Bir dizini siler.